ENPM690 Homework 3 Report

Pranav Deshakulkarni Manjunath — UID: 121162090

March 11, 2026

1 Introduction

This project explores the implementation of robot navigation strategies within a ROS 2 Humble and Gazebo simulation environment. Utilizing a TurtleBot3 Burger, the system demonstrates two modes of operation: manual teleoperation and autonomous boundary-following navigation using the Bug2 algorithm. The simulation is conducted within a custom environment defined by an STL mesh maze, providing realistic geometric constraints for sensor-based navigation.

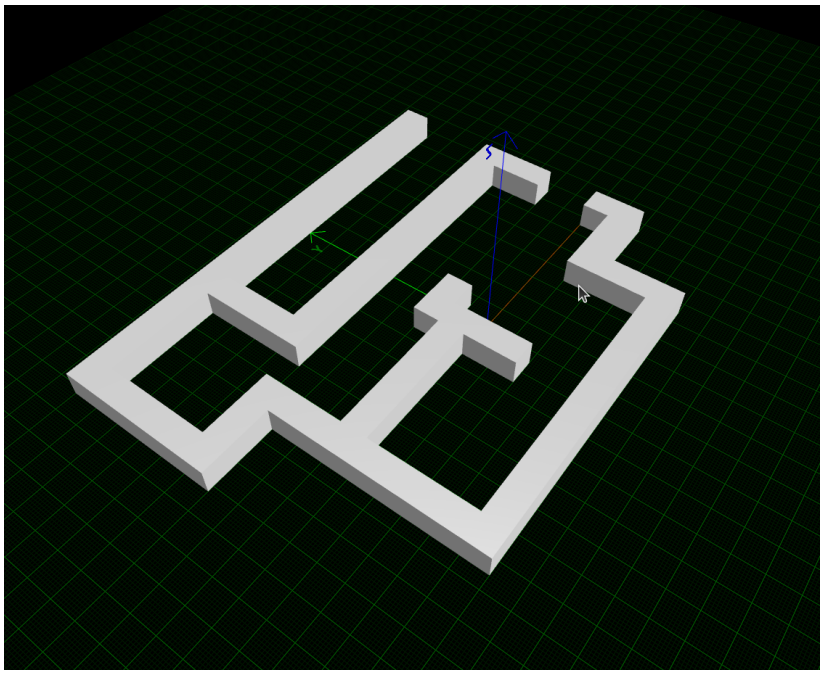


Figure 1: 3D view of the custom STL mesh maze environment.

2 Problem Statement

The objective is to achieve reliable navigation from a starting pose to a target goal within a complex, unknown environment. This involves:

- **Manual Control**: Implementing a responsive teleoperation interface to allow human-in-the-loop navigation.
- **Autonomous Navigation**: Developing a robust algorithm to guide the robot to a target while avoiding obstacles without a global map.
- **Environment Integration**: Ensuring accurate sensor data processing and collision detection within a custom STL-mesh world.

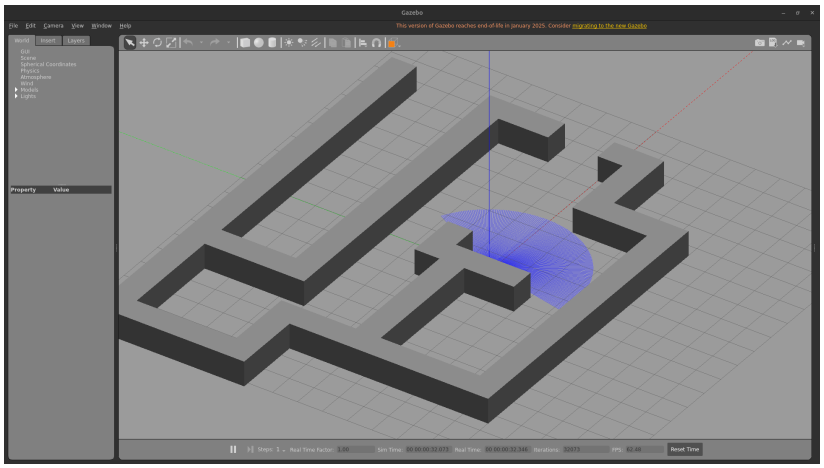


Figure 2: 3D view of the TurtleBot3 spawned in the Gazebo simulation.

3 Methodology

The system architecture decouples user interface, control logic, and simulation parameters.

3.1 Manual Teleoperation

The teleoperation system utilizes a keyboard-based interface with smooth acceleration and deceleration profiles. This design prevents mechanical jitter and robot instability during manual maneuvers. Real-time feedback is provided through a terminal status bar, allowing the operator to monitor target and current velocities.

3.2 Autonomous Behavior: Bug2 Algorithm

For autonomous navigation, the **Bug2 Algorithm** was implemented. It uses a fixed "M-line" (Main-line) connecting the robot's start position to the goal.

- **Goal Seeking**: The robot attempts to move directly along the M-line toward the target.
- **Wall Following**: Upon detecting an obstacle via Lidar, the robot follows the obstacle's perimeter until it re-intersects the M-line at a point closer to the goal than where it first hit.

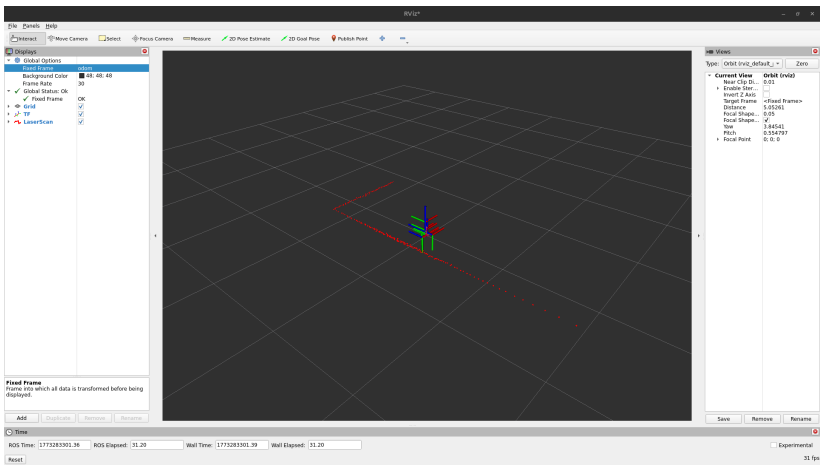


Figure 3: RViz2 visualization showing robot TF frames and active Lidar scan data.

3.3 Parameter Configuration

All navigation parameters are externalized in a YAML configuration file, enabling rapid tuning of maximum speeds, safety distances, and arrival tolerances without source code recompilation.

4 Experiments

Experimental validation focused on tuning the Bug2 algorithm for the STL maze:

- **Safety Zone Tuning**: The `safety_dist` was optimized from 1.0m to 0.4m to allow navigation through narrow corridors while maintaining collision safety.
- **Directional Strategy**: Both left-hand and right-hand rules were tested. In several maze configurations, the choice of side drastically reduced total travel time by avoiding long outer perimeters.

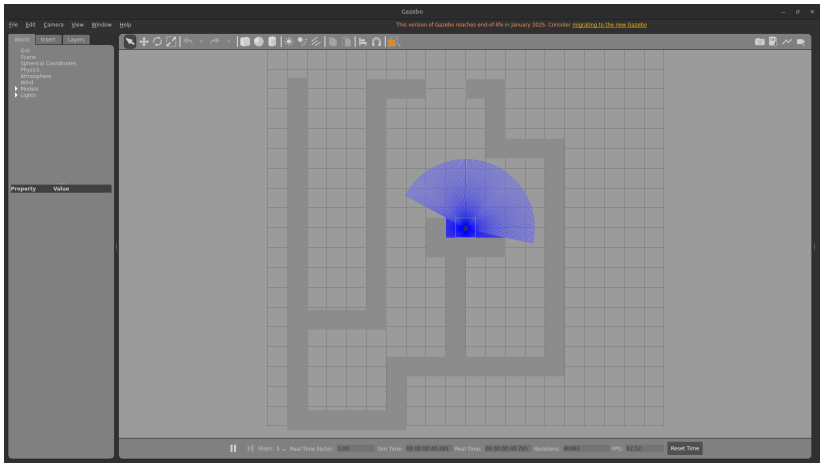
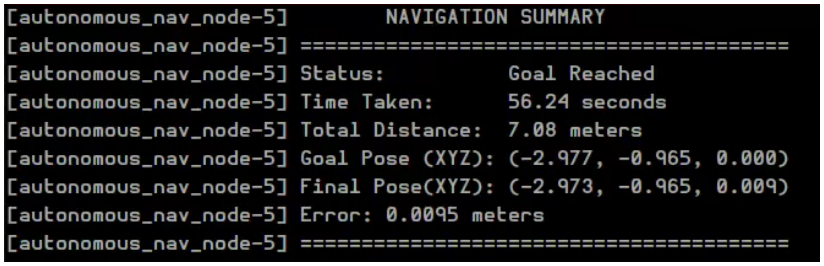


Figure 4: Top-down view of the simulation environment during navigation testing.

5 Results

The resulting system provides stable and reliable navigation. Key performance highlights include:

- **Convergence**: Successful goal arrival in all valid maze configurations.
- **Accuracy**: Average Euclidean error consistently below 0.02m upon goal arrival.



```
[autonomous_nav_node-5]        NAVIGATION SUMMARY
[autonomous_nav_node-5] ========================================
[autonomous_nav_node-5] Status:          Goal Reached
[autonomous_nav_node-5] Time Taken:      56.24 seconds
[autonomous_nav_node-5] Total Distance:  7.08 meters
[autonomous_nav_node-5] Goal Pose (XYZ): (-2.977, -0.965, 0.000)
[autonomous_nav_node-5] Final Pose(XYZ): (-2.973, -0.965, 0.009)
[autonomous_nav_node-5] Error: 0.0095 meters
[autonomous_nav_node-5] ========================================
```

Figure 5: Automated navigation summary showing mission statistics and final positioning error.

The combination of responsive manual control and a refined Bug2 implementation ensures the robot can navigate complex 3D environments efficiently and accurately.